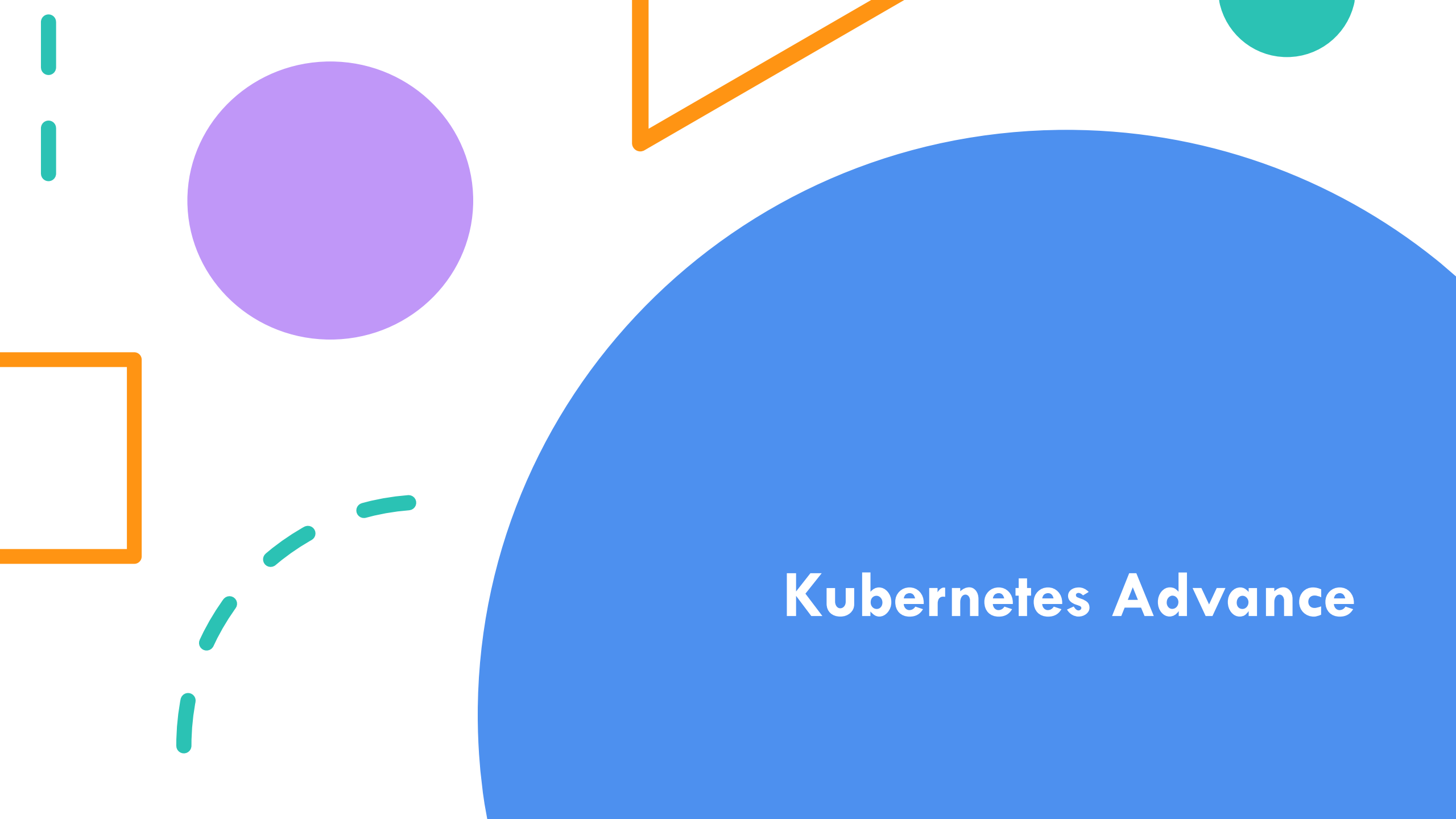




Kubernetes Advance

A large blue circle on the left side of the slide, with a smaller purple circle at its bottom-left edge.

Today's Lesson

K8s workload

K8s resource management

Ingress



K8s Workloads

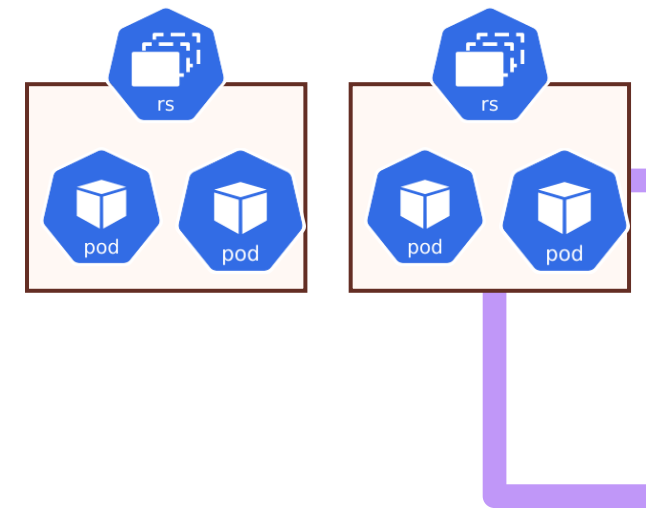
Replica Set

What is a ReplicaSet?

- ReplicaSet is one of the workload on K8s
- A **ReplicaSet (RS)** ensures that a desired number of Pod replicas are running at all times.
- **Self-healing:** If a Pod crashes or is deleted, a new one is created automatically.

Use Cases

- Ensuring high availability – keeping a fixed number of identical Pods running.
- Scaling up or down an application manually.
- Managing stateless, identical Pods.



Replica Set

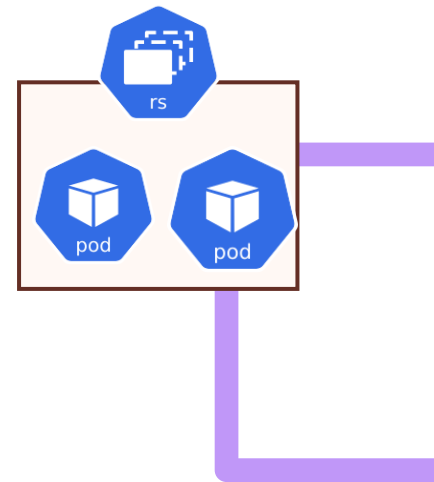
Apply the YAML:

- `kubectl apply -f replicaset.yaml`

Verify:

- `kubectl get rs`
- `kubectl get pods -l app=myapp`

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: my-replicaset
  labels:
    app: myapp
spec:
  replicas: 2
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
        - name: myapp-container
          image: nginx:latest
          ports:
            - containerPort: 80
```



Replica Set

Apply the YAML:

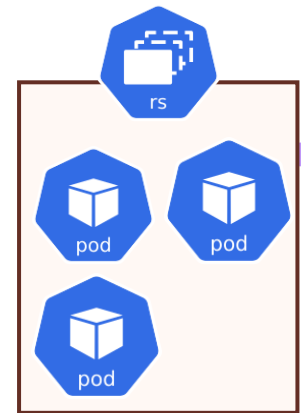
- `kubectl apply -f replicaset.yaml`

Verify:

- `kubectl get rs`
- `kubectl get pods -l app=myapp`

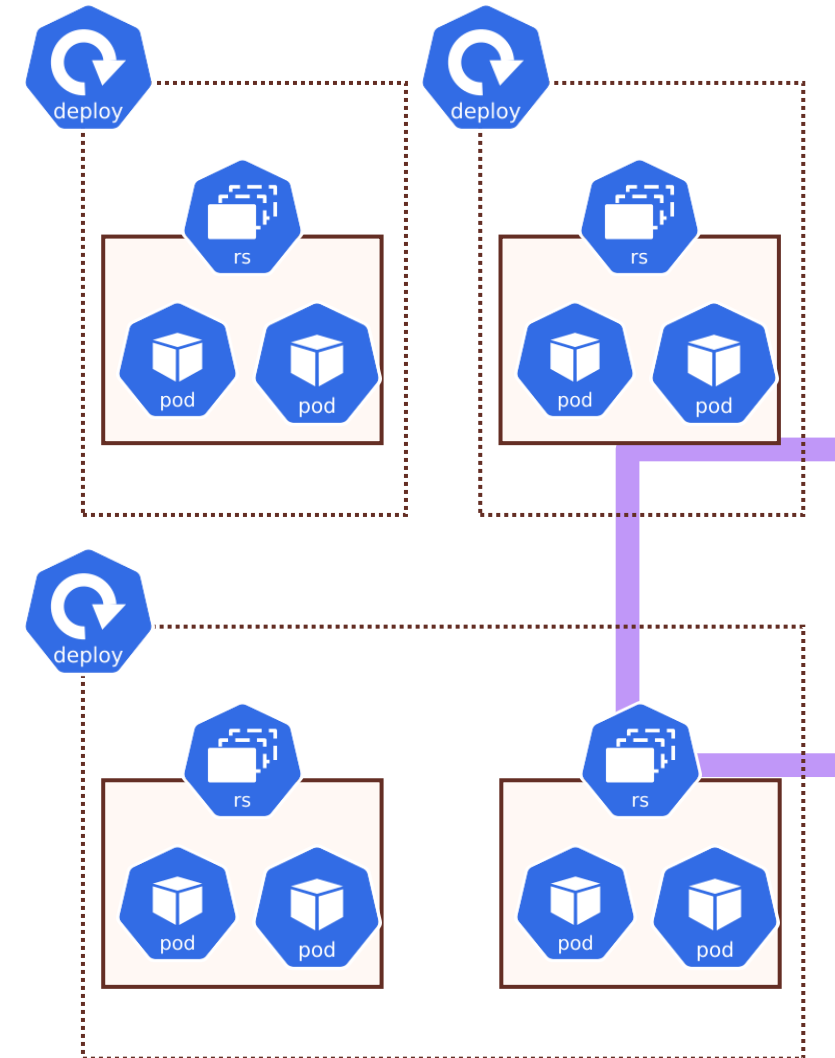
We can control number of pod by adjusting **replicas**

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: my-replicaset
  labels:
    app: myapp
spec:
  replicas: 3 
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
        - name: myapp-container
          image: nginx:latest
          ports:
            - containerPort: 80
```



Deployment

- A **Deployment** in Kubernetes is a higher-level **controller** that manages **ReplicaSets** (and in turn, Pods).
- It provides declarative updates for Pods and ReplicaSets – meaning you describe *what you want*, and Kubernetes ensures the cluster matches that desired state.

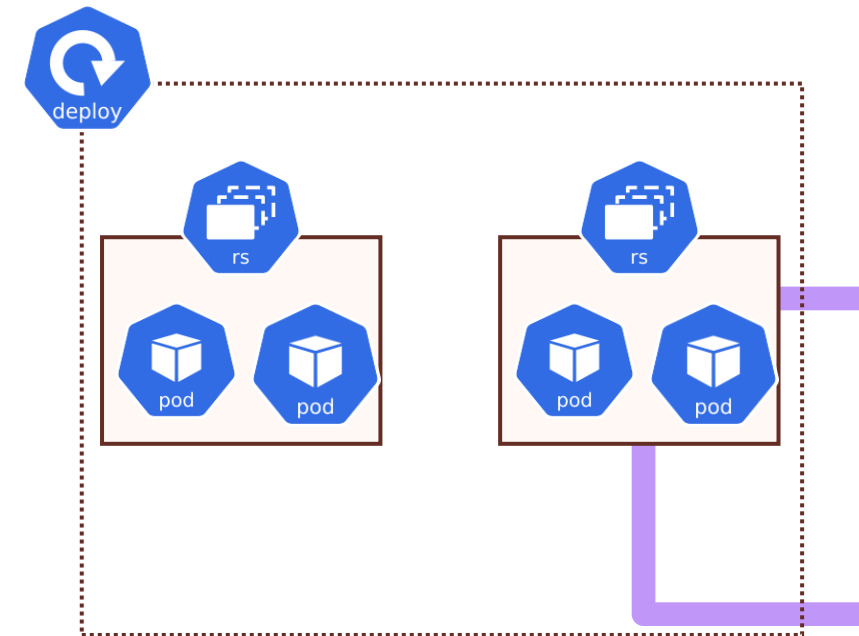


Deployment

- Kubernetes automatically creates and manages ReplicaSets based on your Deployment manifest.
- If you update the deployment (for example, change the image version), Kubernetes performs a **rolling update** – starting new Pods and gracefully terminating old ones.

Use Cases of Deployment

- **Rolling updates:** Updating applications to a new version without downtime.
- **Rollbacks:** Reverting to a previous version in case of a bad deployment.
- **Scaling apps:** Easily scaling up/down Pods.
- **Ensuring HA (High Availability):** Keeps desired Pods running even if nodes fail.
- **Stateless applications:** Running multiple identical replicas (like NGINX, frontend apps, APIs, etc.).



Deployment

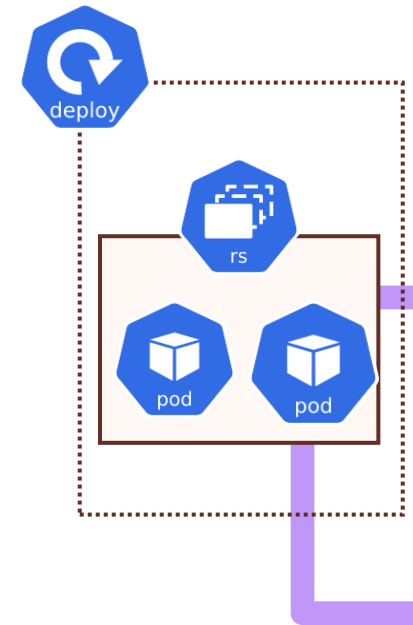
Apply the YAML:

- `kubectl apply -f deployment.yaml`

Verify:

- `kubectl get deployments`
- `kubectl get rs`
- `kubectl get pods -l app=myapp`

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-deployment
  labels:
    app: myapp
spec:
  replicas: 2
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
        - name: myapp-
          container
            image: nginx:latest
            ports:
              - containerPort: 80
```

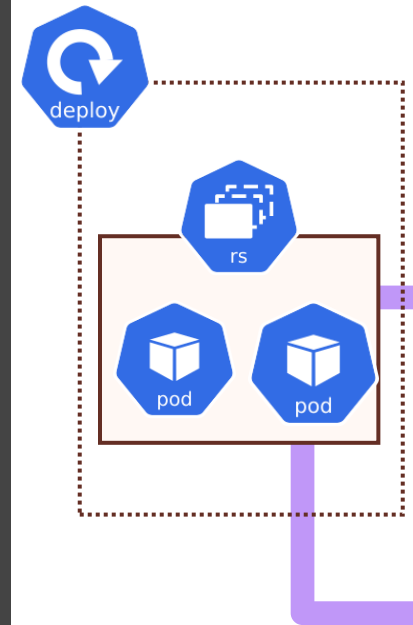


Deployment

Common Deployment Operations

- Scale deployment
 - `kubectl scale deployment my-deployment --replicas=5`
- Update image
 - `kubectl set image deployment/my-deployment myapp-container=nginx:1.25`
- Rollback to previous version:
 - `kubectl rollout undo deployment/my-deployment`
- Check rollout status:
 - `kubectl rollout status deployment/my-deployment`

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-deployment
  labels:
    app: myapp
spec:
  replicas: 2
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
        - name: myapp-container
          image: nginx:latest
          ports:
            - containerPort: 80
```



Deployment vs ReplicaSet

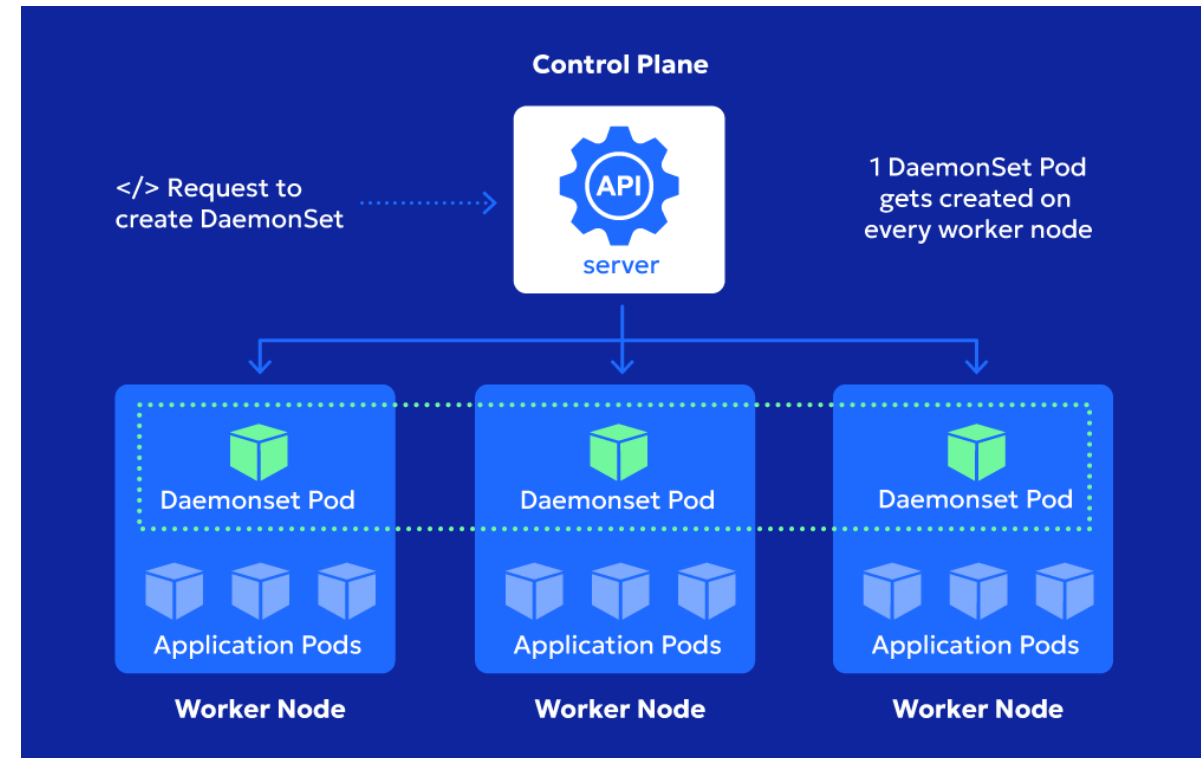
Feature	ReplicaSet	Deployment
Ensures desired pods are running	✓	✓
Rolling updates	✗	✓
Rollbacks	✗	✓
Automatically manages ReplicaSets	✗	✓
Recommended for production	✗	✓

Daemon Set

A **DaemonSet** automatically deploys and manages Pods across nodes.

When new nodes are added to the cluster, the DaemonSet ensures that the Pod is automatically scheduled there.

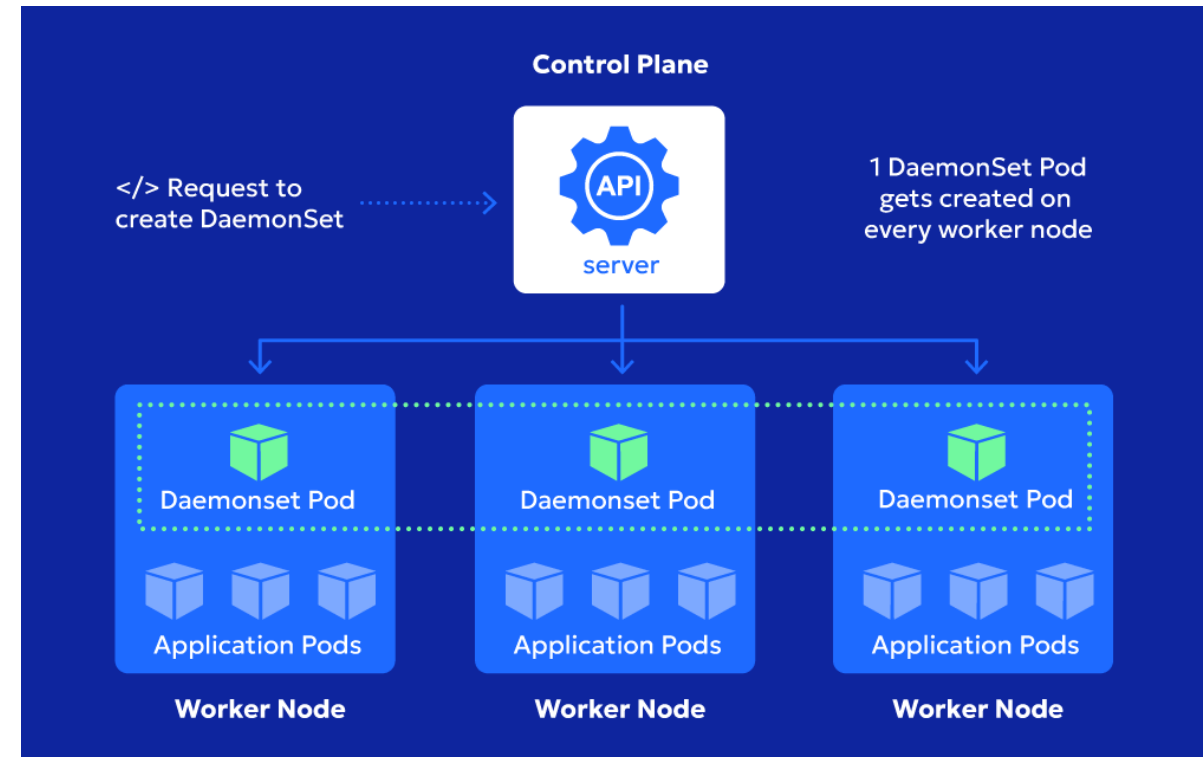
When nodes are removed, those Pods are also cleaned up.



Daemon Set

Common Use Cases

- Cluster storage daemons (e.g., fluentd, logstash, or other log collectors)
- Node monitoring agents (e.g., node-exporter, Datadog agent)
- Network-related pods (e.g., calico, flannel, or CNI plugins)



Daemon Set

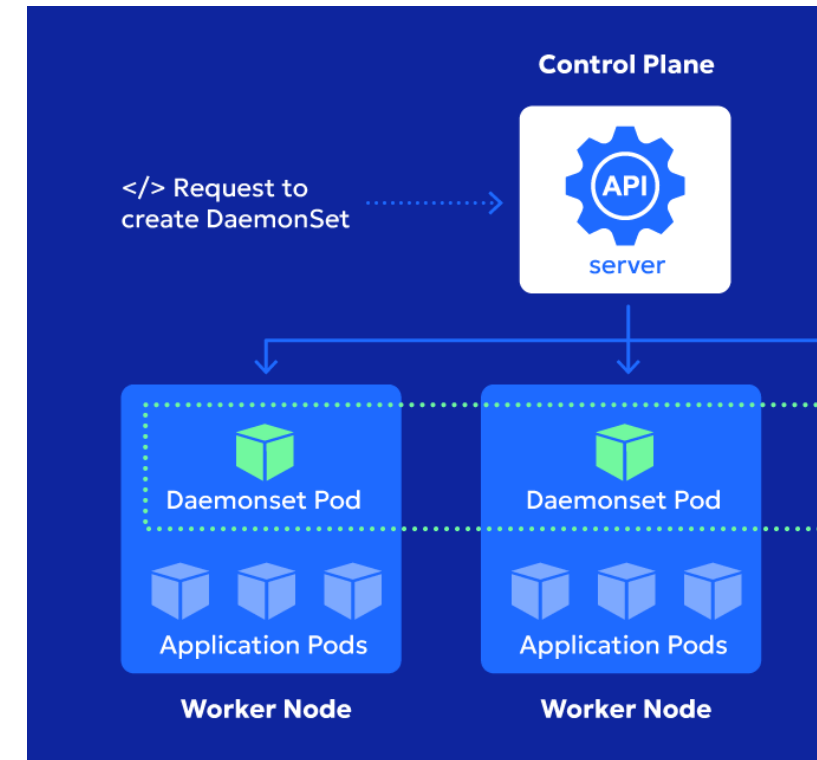
Apply the YAML:

- `kubectl apply -f daemonset.yaml`

Verify:

- `kubectl get daemon`
- `Kubectl get pod -l app=fluentd`

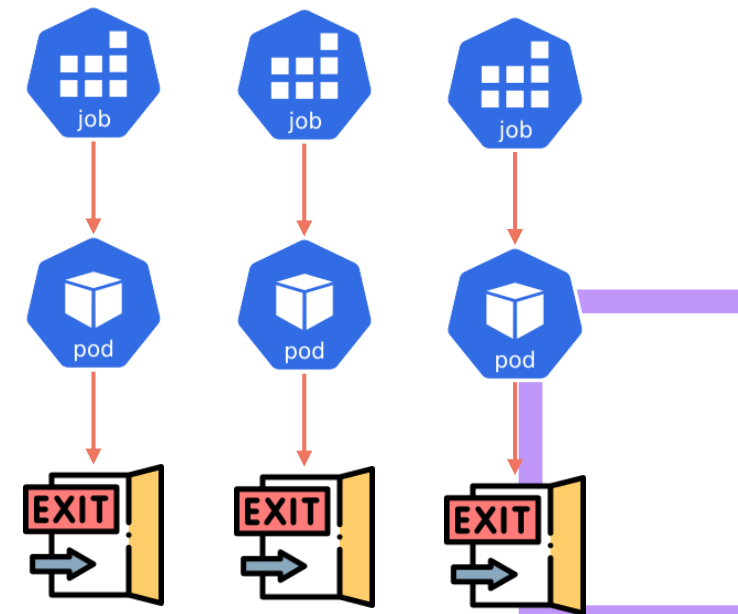
```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: fluentd-daemonset
  namespace: kube-system
spec:
  selector:
    matchLabels:
      app: fluentd
  template:
    metadata:
      labels:
        app: fluentd
    spec:
      containers:
        - name: fluentd
          image: fluent/fluentd:v1.15
          resources:
            limits:
              memory: 200Mi
            requests:
              cpu: 100m
              memory: 200Mi
          volumeMounts:
            - name: varlog
              mountPath: /var/log
      volumes:
        - name: varlog
          hostPath:
            path: /var/log
```



Job

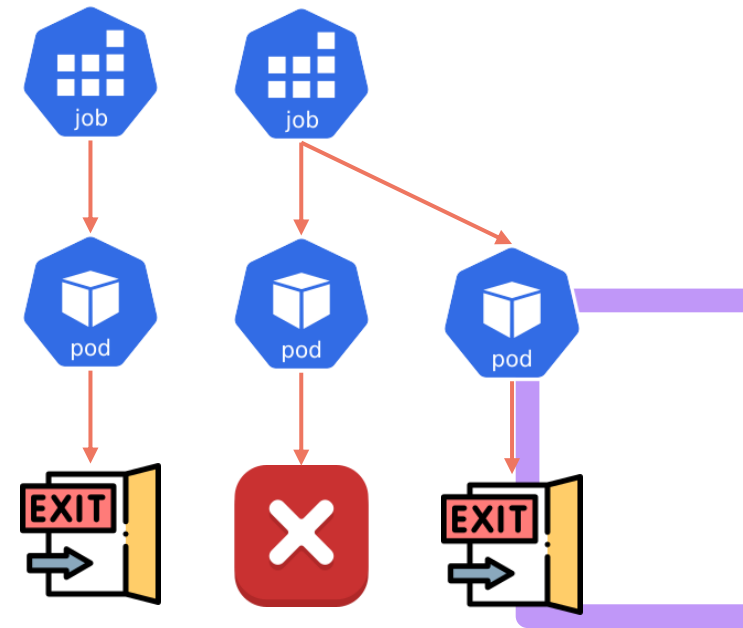
- A **Job** in **Kubernetes (k8s)** is a workload resource used to run **batch or adhoc tasks**—that is, tasks that **complete successfully once** instead of running continuously like Deployments or DaemonSets.

Use Case	Description
Batch processing	Run a short-lived computation or background job like data analysis or report generation
Data import/export	Move data between systems or run database backup tasks
Automation tasks	Run periodic cleanups or maintenance scripts
Running once-off scripts	For example, database migrations on startup



Job

- Pod exit with status code **0** indicate with job success
- If the pod exit with status code different than **0** will be treat as error and will be re-schedule



Job

Apply the YAML:

- `kubectl apply -f job.yaml`

Verify:

- `kubectl get job`

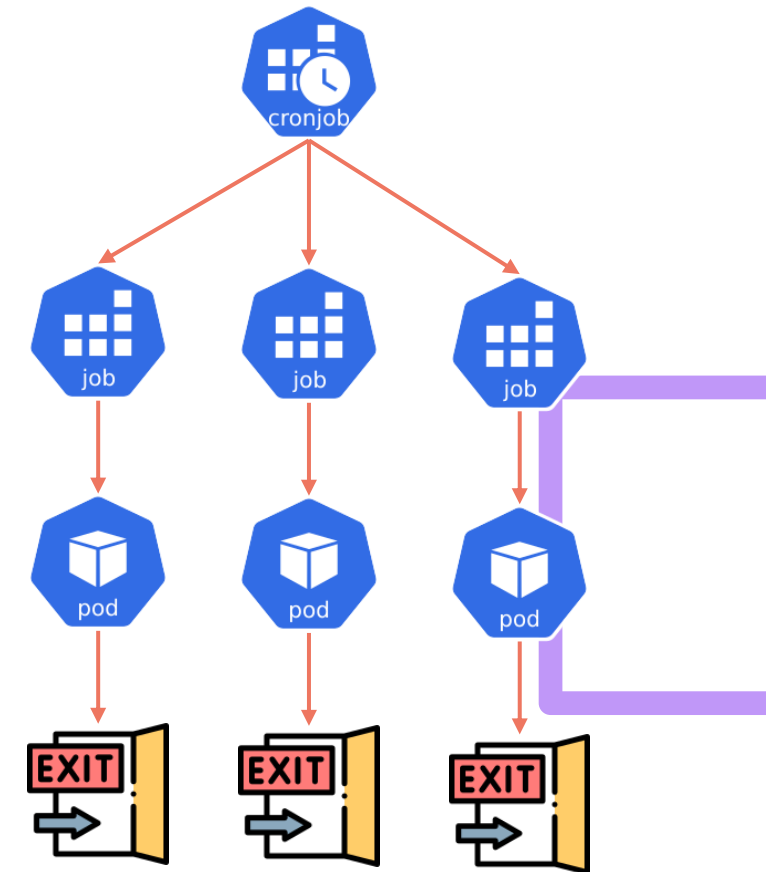
```
apiVersion: batch/v1
kind: Job
metadata:
  name: hello-job
spec:
  template:
    spec:
      containers:
        - name: hello
          image: busybox
          command: ["echo", "Hello from the
Kubernetes Job!"]
          restartPolicy: Never
      backoffLimit: 3
```



CronJob

- A **CronJob** creates a **Job** at a specific time or interval following a **cron schedule expression**. Each time the schedule is triggered, Kubernetes spins up a new Pod (via a Job) to perform the task.

Use Case	Description
Database backups	Run nightly or hourly backups automatically
Log rotation/cleanup	Clear or archive logs at regular intervals
Email or report generation	Send daily/weekly reports
Data synchronization	Periodic data imports/exports between services
System health checks	Execute automated verification scripts periodically



CronJob

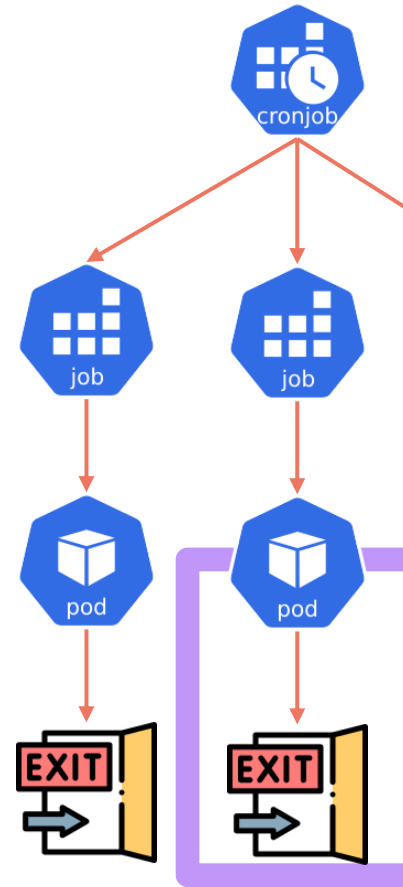
Apply the YAML:

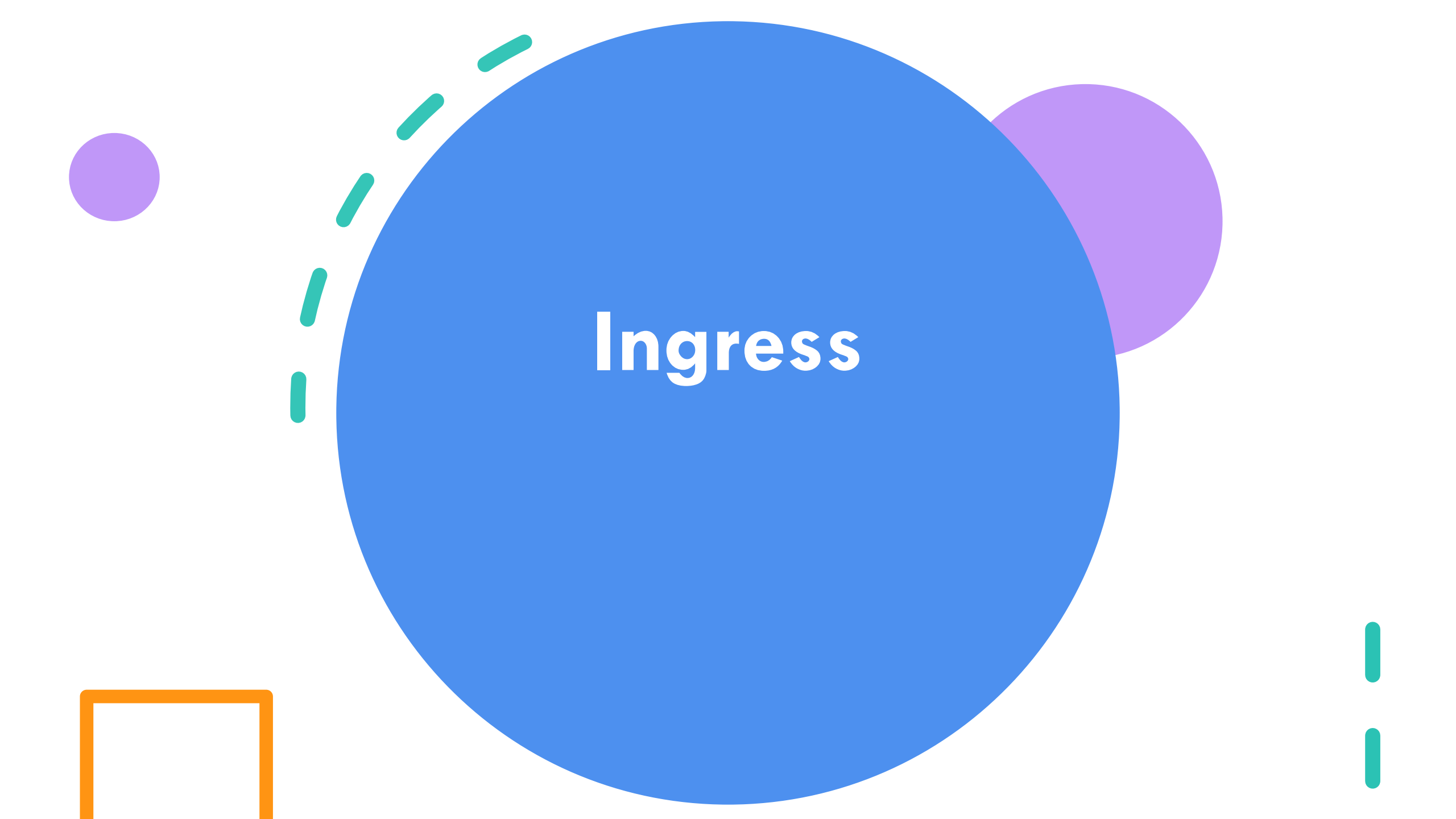
- `kubectl apply -f cronjob.yaml`

Verify:

- `kubectl get cronjob`

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: hello-cronjob
spec:
  schedule: "*/2 * * * *" # Every 2
minutes
jobTemplate:
  spec:
    template:
      spec:
        containers:
          - name: hello
            image: busybox
            command: ["sh", "-c", "date;
echo Hello from Kubernetes CronJob!"]
            restartPolicy: OnFailure
```



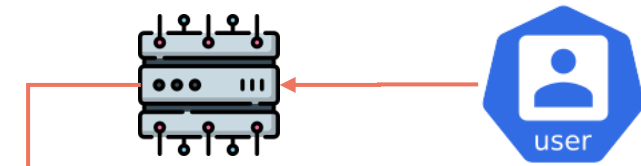


Ingress

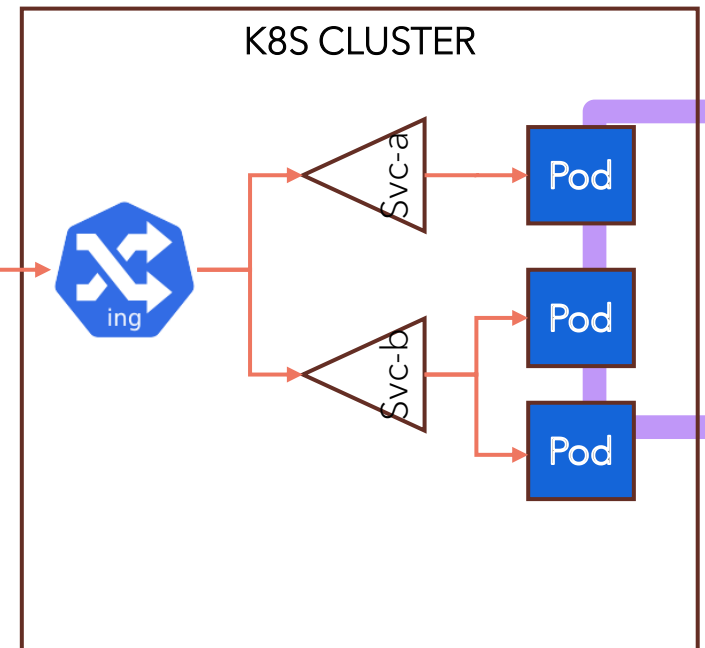
Ingress

- An **Ingress** defines rules for routing external HTTP/HTTPS traffic to internal Services. It lets you expose multiple Services through a **single external IP** and manage routing, SSL/TLS, and virtual hosts easily.

Load
balancer



K8S CLUSTER



Feature	Benefit
Consolidates entry point	Expose multiple services under one LoadBalancer/IP
Path- or host-based routing	Route traffic by URL path or domain
TLS termination	Handle HTTPS certificates in one place
Rewrite / redirect rules	Simplify complex routing
Central management	Easier to manage than multiple NodePorts or LoadBalancers

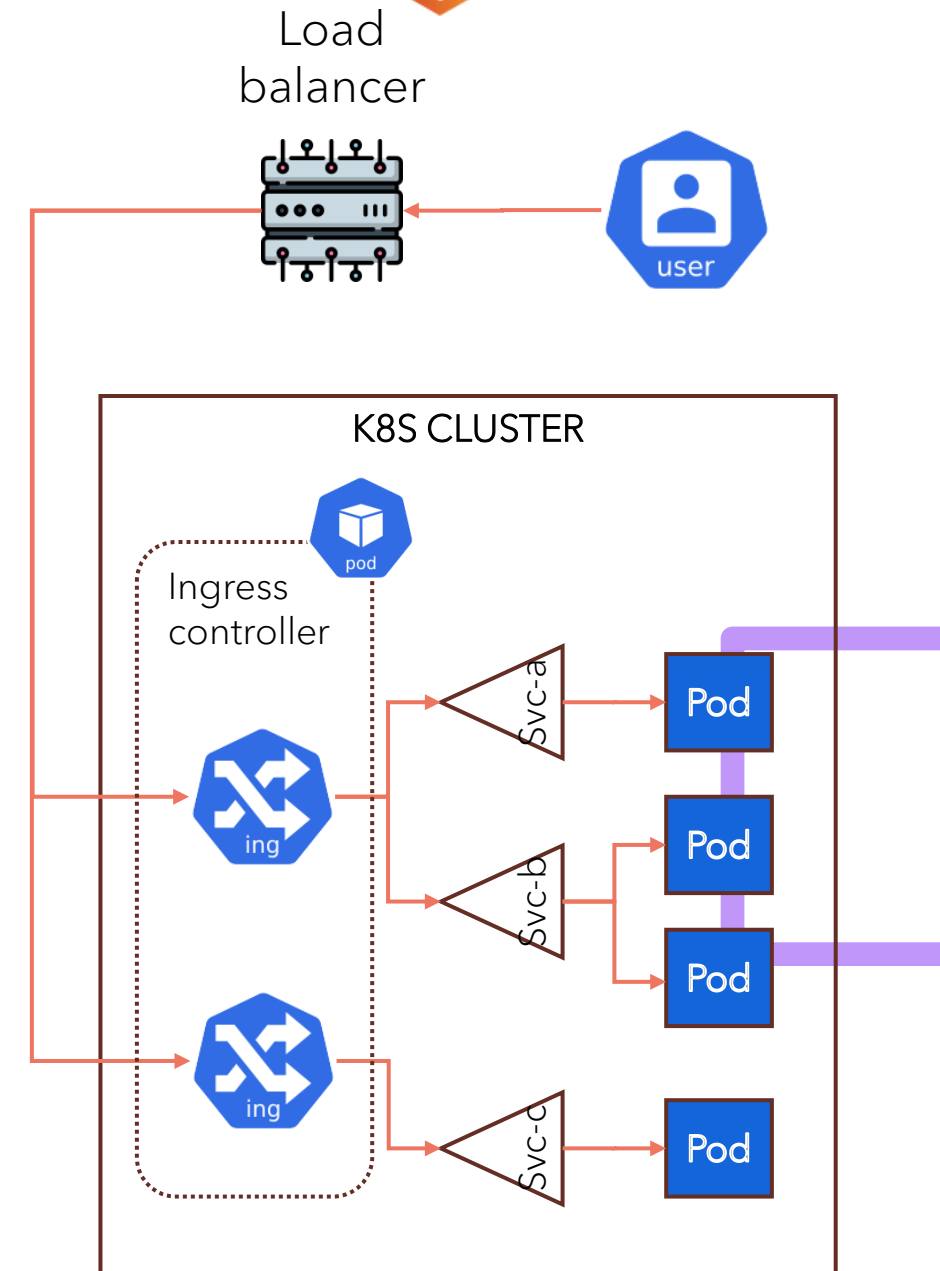
Ingres & Ingress Controller

- An Ingress Controller is the **actual component (software)** that **implements** the routing rules defined in your **Ingress** resources.
- Think of it like this:
 - Ingress = The “rules” (configuration)
 - Ingress Controller = The “traffic police” that enforces those rules and routes the traffic



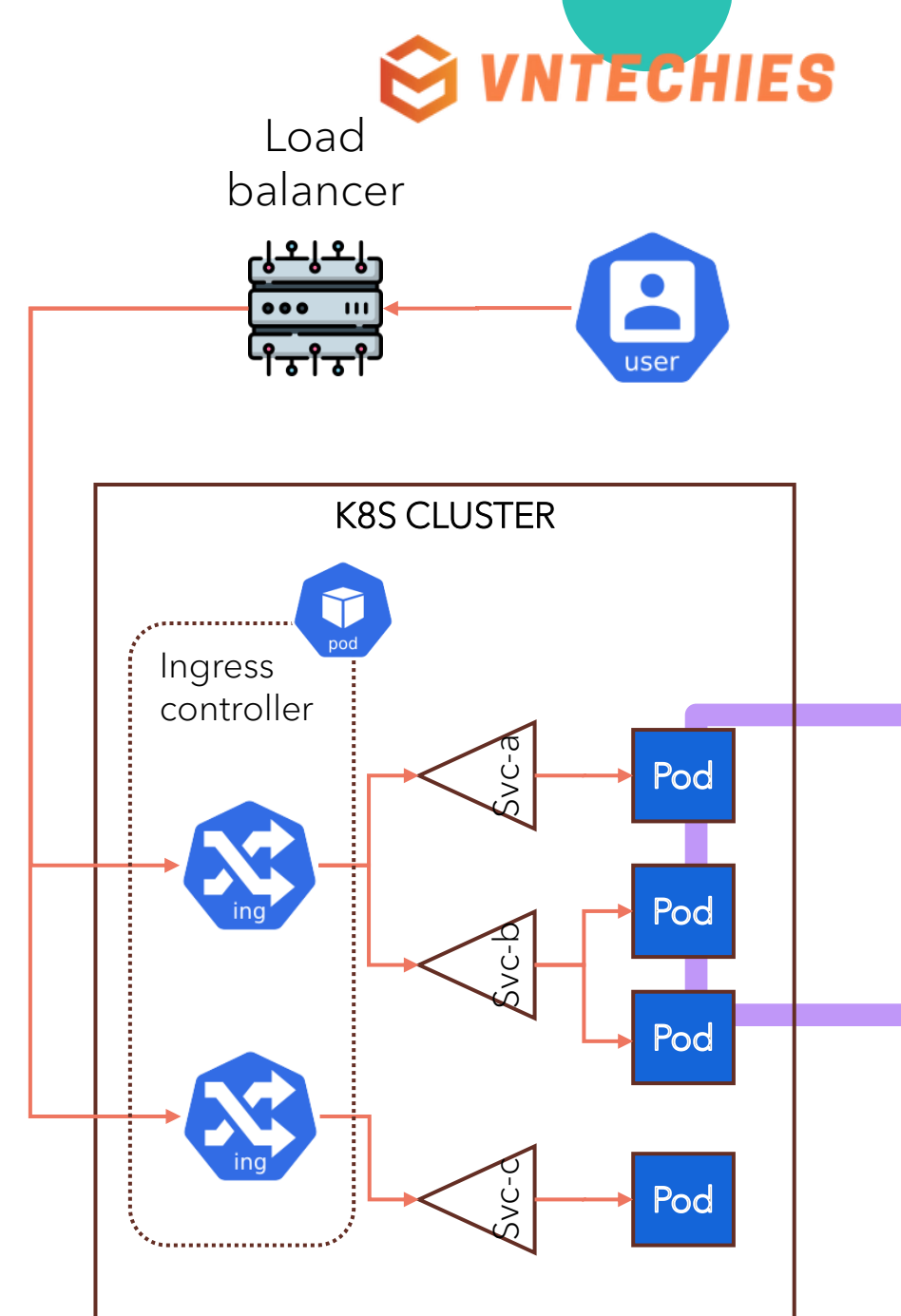
Ingres & Ingress Controller

Component	Purpose
Ingress	Says: "Send /api → Service A, /app → Service B"
Ingress Controller	Actually listens on the cluster's external IP, parses those rules, and routes traffic accordingly



Route traffic using Ingress

- We need to have Ingress Controller installed
- K8s Pod and Service is created
- Ingress is created to route traffic to k8s Service



Route traffic using Ingress



```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: multi-service-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  rules:
    - host: example.com
      http:
        paths:
          - path: /a
            pathType: Prefix
            backend:
              service:
                name: svc-a
                port:
                  number: 80
          - path: /b
            pathType: Prefix
            backend:
              service:
                name: svc-b
                port:
                  number: 80
```

Route for svc-a

Route for svc-b

Svc-a

Svc-b

```
apiVersion: v1
kind: Service
metadata:
  name: svc-a
spec:
  selector:
    app: app-a
  ports:
    - port: 80
      targetPort: 8080
```

```
---
apiVersion: v1
kind: Service
metadata:
  name: svc-b
spec:
  selector:
    app: app-b
  ports:
    - port: 80
      targetPort: 8080
```

Route traffic using Ingress



Section	Description
apiVersion	Current Ingress API version (networking.k8s.io/v1)
ingressClassName	Specifies which controller (e.g., NGINX) should handle this Ingress
host	DNS name used by clients (can be your domain)
paths	Define routes: /a and /b
service.name	The internal Kubernetes Service that receives the traffic
port.number	The target Service port

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: multi-service-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  rules:
    - host: example.com
      http:
        paths:
          - path: /a                # Route for svc-a
            pathType: Prefix
            backend:
              service:
                name: svc-a
                port:
                  number: 80
          - path: /b                # Route for svc-b
            pathType: Prefix
            backend:
              service:
                name: svc-b
                port:
                  number: 80
```



Practice Lab